AI 构图辅助相机 AI 引擎零基础自学教材

基于 /Volumes/桌面/AI-camera 项目的 v9 student、SigLIP2 teacher、template index 与 Android ONNX Runtime Mobile 接入口径

本教材面向项目 AI 负责人。目标不是让你从第一天就手写完整 App 或重写全部训练框架，而是让你真正理解模型训练、评估、迭代、导出和交付模型包所需的底层知识。

教材采用三层讲法：第一层讲人话和摄影模板推荐类比；第二层讲机器学习、数据结构、模型格式和 Android runtime 的底层原理；第三层落到本项目真实文件和核心代码，解释关键语法、函数调用、数据字段和验收方法。

> 范围说明：本教材覆盖 AI 负责人必须掌握的主链路和 v9 首发相关核心代码。全仓库每个辅助脚本逐行解释可以继续扩展成第二册；本册优先保证你能读懂、验收和决策当前商业路线。

本项目当前事实

项目事实	当前值
商业路线	蒸馏已有 teacher 模型，形成自有端侧 embedding student 模型
teacher	google/siglip2-base-patch16-224，768 维，用于离线质量上限和蒸馏
Android 首发候选	tiny_cnn_v9_wide_topk_siglip2_512
端侧模型文件	student_model.static_int8.onnx
runtime	ONNX Runtime Mobile
输入	RGB、float32、NCHW、[1,3,224,224]
输出	512 维 L2 normalized embedding
候选库	new_batch v9 static int8 ONNX index，728 条，尚未合并为正式 visual-clean
正式库边界	visual-clean 104 条正式非人像库，不能被 new_batch 污染
离线质量	v9 static int8 ONNX top5 同场景 96.15%，top5 同类型 98.21%
端侧预算	p50 <= 80ms，p90 <= 150ms，模型 <= 5MB，500ms-1000ms 抽帧

学习地图

1. 项目全景：素材、审核、标注、metadata、teacher embedding、student 训练、index、Android runtime。
2. 零基础计算机知识：文件路径、JSON/JSONL、二进制 float32、命令行、Python 包。
3. 图片和 tensor：RGB、尺寸、resize、归一化、NCHW、batch。

4. embedding：向量、L2 normalize、cosine similarity、dot product、topK、margin。
5. 模型结构：CNN、Conv2d、BatchNorm、激活函数、stride、pooling、Linear、参数量。
6. 蒸馏训练：teacher/student、PCA 投影、cosine loss、relation loss、listwise、hard negative。
7. 索引实现：metadata、embedding 文件、keys、USearch、exact fallback、维度校验。
8. 评估指标：top1/top3/top5、同场景、同类型、构图、光线、低 margin、近重复。
9. 导出和量化：checkpoint、TorchScript、ONNX、dynamic int8、static int8、校准、manifest。
10. Android 交付：ONNX Runtime Mobile、抽帧、top5 rerank、稳定策略、真机验收。
11. 负责人日常：看哪些数据，问 Android 队友什么，什么时候 v9 可首发，什么时候必须 v10。

第 1 章 项目全景：这个 AI 引擎到底在做什么

这个项目不是完整相机 App，也不是图像生成项目。它是一个摄影模板推荐引擎：用户打开相机，对准当前画面，端侧模型把当前画面转换成 embedding，然后用这个 embedding 在模板库里检索相似模板，App 再把命中的模板 metadata 转成构图辅助线和提示。

你作为 AI 负责人，最重要的是管住三件事：数据是否可信，模型是否真的学到 teacher 的推荐空间，交付给 Android 的模型和 index 是否同版本且可验收。

完整链路

```
素材图片
  -> raw manifest
  -> SHA / 尺寸 / 比例 / 重复校验
  -> 人像姿态筛选
  -> 人工质量审核
  -> CV preprocessing jobs
  -> 视觉标注 annotations.jsonl
  -> template_metadata.jsonl
  -> SigLIP2 teacher embedding
  -> 蒸馏数据集 train / val / test
  -> v5 / v6 / v8 / v9 student 训练对比
  -> v9 FP32 ONNX 导出
  -> v9 static int8 ONNX 量化
  -> v9 static int8 ONNX template embedding
  -> template index
  -> Android ONNX Runtime Mobile
  -> topK 检索
  -> top5 + 规则重排
  -> metadata 生成辅助线/提示
```

每个产物的职责

产物	职责	你要验收什么
manifest.jsonl	原始候选照片清单	数量、路径、hash、授权、审核状态
annotations.new_batch.jsonl	视觉标注结果	schema 通过，不编造，template_value/aesthetic_score 合理
template_metadata.*.jsonl	端侧模板说明书	photo_type、scene、composition、lighting、embedding 字段齐全
teacher embedding	SigLIP2 生成的高质量目标空间	768 维、L2 ready、teacher index 质量作为上限
checkpoint.best.pt	训练后最优 student 权重	best epoch、val/test cosine、参数配置
student_model.static_int8.onnx	Android 首发端侧模型	大小、sha256、输入输出、int8 质量回归
template_index/	模板检索库	count、dimension、keys/metadata/embeddings 顺序一致
quality_report	离线检索质量报告	top5 同场景、同类型、问题样本、低 margin

负责人边界

- 你必须理解模型和 index 的版本关系。v9 static int8 ONNX 是 512 维，不能配 256 维 visual-clean index。
- 你必须理解推荐逻辑。首版 App 要 top5 + 规则重排，不应直接把 ANN top1 当最终模板。
- 你必须看得懂评估。v9 static int8 ONNX top5 同场景 96.15% 可以进入 App 接入和灰度，但缺真机数据时不能宣布完整商用默认模型。
- 你不必亲自写完整 CameraX 链路，但必须能验收 Android 队友的预处理、抽帧、ONNX 输出和稳定策略。

本章自测

1. v9 模型输出的是辅助线坐标吗？如果不是，输出什么？
2. 为什么模型和 template index 必须同版本发布？
3. visual-clean 和 new_batch 的产品边界是什么？
4. 为什么没有 Android 真机 p50/p90 数据时，不能说 v9 已经是完整商用默认模型？

第 2 章 零基础计算机知识：文件、JSONL、二进制和命令行

路径和目录

路径是文件在电脑里的地址。本项目根目录是 /Volumes/桌面/AI-camera。你看到的 tools/preprocessed/student_models/... 是相对路径，意思是从项目根目录往下找。

AI 负责人必须会看路径，因为一次错误的路径就可能让训练脚本读到旧数据、让 Android 拿到错模型、让 index 维度混乱。

目录	含义
tools/build_template_db/	构建数据、embedding、index、训练和导出的工具脚本
tools/raw_photos/	原始候选和人工审核 manifest
tools/preprocessed/	CV 预处理、annotation、embedding、template index 和模型产物
AI引擎设计/	当前决策、路线和进度
App技术设计/	给 Android 队友的接入协议和验收标准

JSON 和 JSONL

JSON 是一种结构化文本。它像 Python 的 dict/list，但保存在文本文件里。JSONL 是 JSON Lines，一行一个 JSON object。项目里的 manifest、metadata、annotation 基本都是 JSONL。

```
{"template_id":"new_batch_00027","photo_type":"portrait","scene_category":"closeup_portrait"}
{"template_id":"new_batch_00028","photo_type":"portrait","scene_category":"closeup_portrait"}
```

代码/语法	解释
{...}	一对花括号表示一个对象。对象里是字段名和值。
"template_id"	字段名必须是字符串，所以用双引号。
:	冒号左边是字段名，右边是字段值。
,	逗号分隔多个字段。
一行一个对象	JSONL 的关键规则。读取时可以一行一行解析，适合大规模数据。

二进制 float32

embedding 是很多小数。如果写成 JSON，会很大、读取慢、精度和格式也容易乱。本项目把 embedding 写成二进制 float32 文件。例如 v9 index 有 728 条模板，每条 512 维，每个 float32 占 4 字节，所以 embeddings 文件大小是 728 * 512 * 4 = 1,490,944 bytes。

```
with embeddings_path.open("wb") as handle:
    for vector in vectors:
        handle.write(struct.pack(f"<{dimension}f", *vector.values))
```

代码/语法	解释
open("wb")	w 表示写入，b 表示 binary 二进制。
for vector in vectors:	逐条模板写入。vector.values 是这条模板的 embedding 数字列表。
f"<{dimension}f"	struct 格式字符串。< 表示小端字节序，dimension 表示写多少个，f 表示 float32。
*vector.values	Python 解包语法，把列表里的 512 个数展开成 512 个参数。
handle.write(...)	把打包后的二进制字节写入 templates.embeddings.f32。

命令行阅读方法

```
cd /Volumes/桌面/AI-camera
python3 tools/build_template_db/validate_template_index.py \
  tools/preprocessed/template_index.new_batch.student-tiny-cnn-v9-static-int8-onnx-512
```

命令行不是黑魔法。第一行进入项目目录。第二行用 Python 运行一个脚本。反斜杠表示命令太长，换行继续写。最后一个参数是要校验的 index 目录。

本章自测

1. JSONL 和 JSON 的区别是什么?
2. 为什么 embedding 更适合写成 float32 二进制，而不是直接写在 JSON 里?
3. 728 条、512 维、float32 的 embeddings 文件理论大小是多少?
4. 路径 tools/preprocessed/... 是绝对路径还是相对路径?

第 3 章 Python 语法速成：读懂项目脚本的最低必备

你不需要先成为 Python 专家，但必须能读懂项目脚本里最常见的语法：import、函数、类型注解、Path、dict/list、for 循环、异常、with、dataclass。

import 和 from import

```
import json
import struct
from pathlib import Path
from dataclasses import dataclass
```

代码/语法	解释
import json	导入 Python 标准库 json，用来读写 JSON/JSONL。
import struct	导入 struct，用来把 float 列表打包成二进制。
from pathlib import Path	从 pathlib 模块导入 Path 类，用对象方式处理文件路径。
from dataclasses import dataclass	导入 dataclass，用来快速定义只存数据的小类。

函数定义和类型注解

```
def load_jsonl(path: Path) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    with path.open("r", encoding="utf-8") as handle:
        for line_number, line in enumerate(handle, 1):
            if not line.strip():
                continue
            value = json.loads(line)
            rows.append(value)
    return rows
```

代码/语法	解释
def load_jsonl(...):	def 定义函数。load_jsonl 是函数名。
path: Path	类型注解，表示参数 path 预期是 Path 对象。运行时不强制，但帮助读代码和 IDE 检查。
-> list[dict[str, Any]]	返回值类型注解：返回一个列表，列表里每个元素是 dict。
rows = []	创建空列表，用来存解析出来的每一行。
with path.open("r", encoding="utf-8") as handle:	打开文件。with 能保证文件用完自动关闭。r 表示读取。
enumerate(handle, 1)	遍历文件每一行，同时拿到行号。1 表示行号从 1 开始。
if not line.strip(): continue	如果空行就跳过。strip 去掉空白字符。continue 进入下一轮循环。
json.loads(line)	把一行 JSON 字符串解析成 Python dict。
rows.append(value)	把解析结果加入列表末尾。

代码/语法	解释
return rows	返回所有解析结果。

dataclass

```
@dataclass(frozen=True)
class TemplateVector:
    key: int
    record: dict[str, Any]
    values: list[float]
```

代码/语法	解释
@dataclass(frozen=True)	装饰器。让 Python 自动生成初始化函数等。frozen=True 表示创建后不允许修改，降低意外改数据的风险。
class TemplateVector:	定义一个类。这里它只是一个结构化容器。
key: int	模板在 index 里的数字 key。
record: dict[str, Any]	对应的 metadata 记录。
values: list[float]	这条模板的 embedding 数字列表。

异常处理

```
try:
    vectors = load_ready_vectors(args.metadata, args.dimension)
except (OSError, ValueError, KeyError) as exc:
    print(f"error: {exc}", file=sys.stderr)
    return 1
```

代码/语法	解释
try:	尝试执行可能失败的代码。
except (...) as exc:	如果发生指定类型错误，就进入 except。exc 是错误对象。
OSError	文件读写错误，例如文件不存在、权限问题。
ValueError	值不合法，例如维度不对、JSON 格式不对。
KeyError	dict 里缺少必需字段。
file=sys.stderr	把错误打印到标准错误流，便于命令行区分正常输出和错误。
return 1	命令返回码 1 表示失败。返回 0 才表示成功。

本章自测

1. Path 对象解决的是什么问题?
2. with open 和普通 open 相比有什么好处?
3. 类型注解会不会在运行时自动保证数据正确?
4. return 1 在命令行脚本里通常表示成功还是失败?

第 4 章 图片、像素、tensor 和预处理

图片在模型眼里是什么

人看到照片，会说这是人像、风景、食物。模型看到的是数字。RGB 图片可以理解为三张二维表：R 通道、G 通道、B 通道。每个像素每个通道通常是 0 到 255。

v9 端侧模型要求输入是 [batch, 3, 224, 224] 的 float32 tensor。batch 是一次输入几张图，3 是 RGB 三个通道，224x224 是统一尺寸。

为什么必须做预处理

- 相机原图尺寸不固定，模型要求固定 224x224。
- Android 相机帧可能是 YUV，需要转 RGB。
- 照片可能带 EXIF 方向，需要按 rotationDegrees 转正。
- 模型训练时用的是 NCHW 和 [-1,1] 归一化，App 推理必须一致。

项目里的 ONNX 离线预处理代码

```
def load_tensor(record, image_size, base_dir, image_cache_dir):
    photo_id = str(record.get("photo_id"))
    transform = transform_for_split(
        image_size,
        train=False,
        pre_resized=image_cache_dir is not None,
    )
    image_path = resolve_path(str(record.get("source_path") or ""), base_dir)
    with Image.open(image_path) as image:
        image = ImageOps.exif_transpose(image).convert("RGB")
        tensor = transform(image)
    return tensor.numpy().astype(np.float32)
```

代码/语法	解释
photo_id = str(...)	取照片 ID 并转成字符串，后续用于缓存路径和报错定位。
transform_for_split(...)	构造和训练/验证一致的图片变换：resize、转 tensor、归一化等。
train=False	推理阶段不做训练增强，保证结果稳定。
pre_resized=...	如果使用缓存图片，说明图片已经预先 resize 过，transform 会相应调整。
resolve_path(...)	把 metadata 里的相对路径解析成真实文件路径。
Image.open(image_path)	用 PIL 打开图片文件。
ImageOps.exif_transpose(image)	按图片 EXIF 方向自动转正，避免横竖颠倒。
convert("RGB")	统一转成 RGB 三通道。
transform(image)	执行 resize、转 tensor、归一化。
tensor.numpy().astype(np.float32)	把 PyTorch tensor 转成 numpy float32，ONNX Runtime 输入需要这种数组。

Android 侧要一模一样

Android 侧的伪代码必须完成同样的事情：ImageProxy 按 rotationDegrees 转正，转 RGB，resize 到 224x224，写成 NCHW float32 buffer，再用公式 value = (rgb / 255.0 - 0.5) / 0.5 归一化。

```
ImageProxy
  -> rotationDegrees 转正
  -> YUV / Bitmap 转 RGB
  -> resize 224x224
  -> NCHW float32
  -> value = (rgb / 255.0 - 0.5) / 0.5
  -> ONNX Runtime 推理
```

常见致命错误

错误	后果	如何验收
RGB/BGR 搞反	颜色语义错，embedding 漂移	用同一张测试图对比 Python 和 Android 输出 cosine
NCHW/NHWC 搞反	模型能跑但结果基本无意义	检查输入 shape 和 buffer 写入顺序
忘记转正	竖图横读，人像/构图判断乱	测试旋转、前后摄切换
归一化公式不一致	embedding 分布偏移	输出 L2 norm 和离线 cosine 对比
每帧推理	卡顿、发热、耗电	记录实际触发间隔和 preview FPS

本章自测

1. v9 的输入 shape 为什么是 [1,3,224,224]?
2. NCHW 中的 C 是什么?
3. 归一化公式把 0、128、255 大约变成什么?
4. 为什么同一张测试图要对比 Python ONNX 输出和 Android ONNX 输出?

第 5 章 embedding、L2 normalize、cosine、topK 和 margin

embedding 的本质

embedding 是一组数字，表示图片在一个学习出来的语义空间里的位置。这个项目把当前画面和模板照片都变成 embedding，然后比较谁离谁更近。

直接分类会把照片压成少数标签，例如 portrait 或 landscape；embedding 则保留了“像不像”的连续关系。两张都是 food_still_life 的照片，可能一个是俯拍居中，一个是侧光留白，它们的 embedding 仍然可以表达差异。

L2 normalize

```
def l2_normalize(values: np.ndarray) -> np.ndarray:
    norms = np.linalg.norm(values, axis=1, keepdims=True)
    norms = np.maximum(norms, 1e-12)
    return (values / norms).astype(np.float32)
```

代码/语法	解释
np.linalg.norm(values, axis=1)	计算每个 embedding 的长度。axis=1 表示按每一行计算。
keepdims=True	保持二维形状，方便后面 values / norms 自动广播。
np.maximum(norms, 1e-12)	防止长度为 0 导致除以 0。1e-12 是很小的数。
values / norms	把每个向量除以自己的长度，使长度变成 1。
astype(np.float32)	转回 float32，和模型/index 文件格式一致。

cosine similarity

cosine similarity 衡量两个向量方向是否接近。两个 embedding 都 L2 normalize 后，cosine similarity 就等于 dot product，也就是对应维度相乘后求和。

```
def cosine(a: list[float], b: list[float]) -> float:
    return sum(left * right for left, right in zip(a, b))
```

代码/语法	解释
zip(a, b)	把两个列表按位置配对，例如第 1 维和第 1 维配对。
for left, right in ...	依次取出每一对数。
left * right	对应维度相乘。
sum(...)	把所有维度的乘积加起来，得到 dot product。
为什么没有除以长度	因为前面已经 L2 normalize，长度都是 1。

topK 和 margin

topK 是取相似度最高的 K 个候选。top1 是第一名，top5 是前五名。margin 是第一名和第二名相似度的差距。margin 很小时，说明模型不确定，App 不应该频繁切换模板。

```
当前帧 embedding q
  -> 与 728 条模板逐个算 cosine
  -> 排序
  -> top1 / top3 / top5
  -> 如果 top1-top2 margin < 0.01，保持当前模板或要求连续稳定领先
```

为什么首版必须 top5 + 规则重排

v9 static int8 ONNX 的 top1 同场景是 88.87%，top5 同场景是 96.15%。这说明第一名偶尔会错，但正确场景经常在前五里。首版产品要利用 top5 的召回优势，再用 photo_type、scene_category、composition、lighting 和稳定策略做重排。

指标	含义	v9 static int8 ONNX
top1 同场景	第一名是否和 query 同 scene_category	88.87%
top5 同场景	前五名里是否至少有同 scene_category	96.15%
top5 同类型	前五名里是否至少有同 photo_type	98.21%
低 margin	top1/top2 差距小于 0.01 的查询数	270

本章自测

1. 为什么 L2 normalize 后 cosine 可以用 dot product 计算?
2. top5 同场景高于 top1 同场景说明什么?
3. margin 小的时候，App 应该立刻切模板吗? 为什么?
4. 为什么 embedding 比直接分类更适合摄影模板推荐?

第 6 章 metadata：模板说明书和 App 辅助线来源

v9 模型只输出 512 维 embedding，不输出辅助线坐标。App 最终画什么线，主要来自 templates.metadata.jsonl。你可以把 metadata 理解为“每张模板的摄影说明书”。

一条模板记录里最重要的字段

字段	作用
template_id / photo_id	唯一标识，用来追踪模板和问题样本
photo_type	大类：portrait、landscape、still_life
scene_category	更细场景：closeup_portrait、food_still_life 等
annotation.composition.type	构图类型，如 rule_of_thirds、centered_subject
annotation.composition.subject_position	主体位置建议
annotation.composition.horizon_position	地平线位置建议
annotation.lighting.*	光线质量、方向、对比、时间
annotation.camera.*	机位、距离、横竖构图
embedding.dimension	embedding 维度。v9 static int8 ONNX 必须是 512
embedding.path	这条模板 embedding.f32 的路径

metadata 到 App 辅助线

metadata 字段	第一版 App 可以怎么用
composition.type = rule_of_thirds	显示三分线和目标交点
composition.type = centered_subject	显示中心线/居中参考
horizon_position = upper_third/lower_third	显示水平参考线
leading_lines = true	提示用户寻找画面延伸线，不强行画几何线段
negative_space = high	提示保留留白区域
visual_summary	转成用户可读的一句话提示

为什么 metadata 要严格校验

如果 metadata 顺序和 embedding 顺序不一致，index 找到的是 A 的 embedding，但 App 取到 B 的说明书，最终辅助线会错。如果 embedding.dimension 不对，检索空间不一致，分数没有意义。

```
if index < len(metadata) and metadata[index].get("template_id") != template_id:
    errors.append(
        f"item {index}: key template_id {template_id} does not match metadata "
        f"{metadata[index].get('template_id')}"
    )
```

代码/语法	解释
index < len(metadata)	避免 metadata 数量不足时访问越界。
metadata[index].get("template_id")	取同一位置 metadata 的 template_id。get 缺字段时返回 None，不直接崩。
!= template_id	比较 keys.json 里的 template_id 和 metadata 同位置 ID 是否一致。
errors.append(...)	发现错位就记录错误。validate 脚本最后会返回失败。
f"...{template_id}..."	f-string，把变量值插入错误消息，方便定位。

本章自测

1. v9 模型是否直接输出辅助线?
2. App 显示三分线主要依据哪个 metadata 字段?
3. metadata 顺序和 embeddings 顺序错位会造成什么问题?
4. 为什么 embedding.dimension 是模型/index 同版本发布的关键字段?

第 7 章 teacher、student 和 distillation

teacher 是什么

teacher 是大模型老师。本项目 teacher 是 google/siglip2-base-patch16-224。它质量高、语义理解强，但 base 模型太大，不适合作为 Android 首发实时端侧模型。teacher 在这里负责建立高质量 embedding 空间，作为离线质量上限和 student 学习目标。

student 是什么

student 是小模型学生。v9 student 是 tiny_cnn_v9_wide_topk_siglip2_512，参数量 223,680，输入 224px，输出 512 维。它要学习 teacher 判断照片相似性的方式，但要足够小、足够快，能在 Android ONNX Runtime Mobile 上跑。

distillation 是什么

蒸馏不是让 student 重新从零认识世界，而是让 student 模仿 teacher 的 embedding 和排序关系。摄影类比：老师看一堆照片，说哪些照片彼此相似、哪些是容易混淆的负样本；学生反复练习，学会在手机上给出接近老师的判断。

teacher embedding 生成代码

```
def normalize(values: list[float]) -> list[float]:
    magnitude = math.sqrt(sum(value * value for value in values))
    if magnitude == 0.0:
        raise ValueError("embedding magnitude is zero")
    return [value / magnitude for value in values]
```

代码/语法	解释
values: list[float]	输入是一组浮点数，也就是原始 embedding。
sum(value * value for value in values)	把每个维度平方后求和。
math.sqrt(...)	开平方，得到向量长度。
if magnitude == 0.0	防止全零 embedding。全零向量没有方向，不能算相似度。
raise ValueError(...)	主动报错，不允许静默写入坏 embedding。
[value / magnitude for value in values]	列表推导式，把每个维度除以长度，得到 L2 normalized embedding。

为什么 teacher 是 768 维，student 是 512 维

SigLIP2 teacher 输出 768 维。本项目为了端侧体积和速度，把 student 输出设成 512 维。训练时会把 teacher 目标投影到 student 维度，让 student 学习一个更小的近似空间。维度改变后必须重建 index，不能把 768 维 teacher index 和 512 维 student 模型混用。

v9 为什么当前最好

版本	top5 同场景	结论
SigLIP2 teacher	97.53%	离线上限
student v5	94.37%	低于 95% 门槛，被 v9 替代
student v6	93.96%	test cosine 接近，但真实检索较低
student v8	94.23%	低 margin 改善，但召回未提升
student v9	96.57%	当前最佳 FP32 student
student v9 static int8 ONNX	96.15%	Android 首发 runtime 候选

本章自测

1. teacher 在本项目里是 Android 端侧模型吗?
2. student 为什么不能直接用 768 维 teacher index?
3. 蒸馏要学生学习老师的哪两类东西：单张 embedding 还是排序关系?
4. 为什么 v6 test cosine 接近 v9，但仍然不是最佳首发候选?

第 8 章 v9 student 模型结构逐句讲解

本章对应 tools/build_template_db/student_embedding_model.py。你要先读懂模型由哪些层组成，再理解为什么它适合端侧：小 CNN、depthwise separable block、stride 降采样、全局池化、Linear 输出、L2 normalize。

配置对象

```
@dataclass(frozen=True)
class StudentModelConfig:
    architecture: str = "tiny_cnn_v1"
    output_dim: int = 256
    image_size: int = 160
    width_mult: float = 0.75
    dropout: float = 0.1
```

代码/语法	解释
@dataclass(frozen=True)	自动生成初始化函数，并禁止后续修改配置，减少实验记录被改乱。
class StudentModelConfig:	定义模型配置类。
architecture: str = "tiny_cnn_v1"	模型架构名称。v9 使用 tiny_cnn_v1。
output_dim: int = 256	默认输出维度。v9 训练时覆盖为 512。
image_size: int = 160	默认输入尺寸。v9 覆盖为 224。
width_mult: float = 0.75	宽度倍率。越大通道数越多，模型容量和计算量越高。v9 是 1.5。
dropout: float = 0.1	训练时随机丢弃部分特征，降低过拟合。

通道数缩放

```
def scaled(value: int, width_mult: float) -> int:
    return max(8, int(round(value * width_mult / 8.0) * 8))
```

代码/语法	解释
value * width_mult	按宽度倍率放大或缩小通道数。
/ 8.0	准备把通道数对齐到 8 的倍数。移动端计算通常喜欢这种对齐。
round(...)	四舍五入到最近整数。
* 8	乘回去，得到 8 的倍数。
int(...)	转成整数通道数。
max(8, ...)	至少保留 8 个通道，避免模型太窄。

基础卷积块

```
class ConvBnAct(nn.Sequential):
    def __init__(self, in_channels: int, out_channels: int, stride: int) -> None:
        super().__init__(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.Hardswish(inplace=True),
        )
```

代码/语法	解释
class ConvBnAct(nn.Sequential):	定义一个顺序模块。输入会依次经过 Conv2d、BatchNorm、Hardswish。
def **init**(...)	构造函数，创建模块时运行。
in_channels	输入通道数。第一层输入 RGB，所以是 3。
out_channels	输出通道数。通道越多，模型能表达的视觉特征越多。
stride	滑动步长。stride=2 会让特征图宽高减半。
super().**init**(...)	调用父类 nn.Sequential 的构造函数，把子层按顺序注册进去。
nn.Conv2d(...)	二维卷积层，用小窗口提取局部视觉模式。
kernel_size=3	卷积核是 3x3。
padding=1	边缘补 1 圈，使 stride=1 时宽高不变。
bias=False	卷积不使用 bias，因为后面 BatchNorm 已经有偏置能力。
nn.BatchNorm2d(out_channels)	对每个通道做归一化，让训练更稳定。
nn.Hardswish(inplace=True)	激活函数，引入非线性。inplace=True 表示尽量原地计算，省内存。

DepthwiseSeparableBlock

```
self.block = nn.Sequential(
    nn.Conv2d(in_channels, in_channels, kernel_size=3, stride=stride,
              padding=1, groups=in_channels, bias=False),
    nn.BatchNorm2d(in_channels),
    nn.Hardswish(inplace=True),
    nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False),
    nn.BatchNorm2d(out_channels),
    nn.Hardswish(inplace=True),
)
self.use_residual = stride == 1 and in_channels == out_channels
```

代码/语法	解释
groups=in_channels	关键点。每个输入通道单独卷积，这就是 depthwise convolution，计算量小。
kernel_size=1	1x1 卷积，用来混合通道信息，也叫 pointwise convolution。
self.use_residual = ...	只有输入输出形状一致时才允许残差连接。
stride == 1	stride=1 才不会改变宽高。

代码/语法	解释
in_channels == out_channels	通道数也必须一致，否则 inputs 和 output 不能相加。

forward：真正的数据流

```
def forward(self, inputs: torch.Tensor) -> torch.Tensor:
    output = self.head(self.features(inputs))
    return F.normalize(output, p=2, dim=1)
```

代码/语法	解释
def forward(...)	PyTorch 模型的前向计算函数。调用 model(images) 时实际执行它。
inputs: torch.Tensor	输入是 tensor，v9 期望 [batch,3,224,224]。
self.features(inputs)	通过卷积特征提取网络，得到高层视觉特征。
self.head(...)	Flatten、Dropout、Linear，把视觉特征压成 output_dim 维。v9 是 512。
F.normalize(..., p=2, dim=1)	对每个样本的 512 维向量做 L2 normalize。dim=1 表示沿 embedding 维度归一化。

为什么这个模型适合 Android

- 输入只有 224x224，比大图计算少。
- 使用 depthwise separable block，减少卷积计算。
- 逐步 stride=2 降采样，快速缩小空间尺寸。
- 最终输出 512 维 embedding，不做复杂检测头。
- static int8 ONNX 后约 301,677 bytes，小于 5MB 预算。

本章自测

1. width_mult 会影响什么?
2. groups=in_channels 为什么能减少计算?
3. 残差连接为什么要求 stride=1 且通道数相同?
4. forward 最后一行为什么要 normalize?

第 9 章 蒸馏训练损失：cosine、relation、listwise、hard negative

本章对应 tools/build_template_db/train_student_embedding.py。训练不是只让 student 的单张 embedding 接近 teacher，还要让 student 学会 teacher 对一批照片的相似关系和排序习惯。

cosine_loss：单张图片对齐

```
def cosine_loss(prediction: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    prediction = F.normalize(prediction, p=2, dim=1)
    target = F.normalize(target, p=2, dim=1)
    return 1.0 - F.cosine_similarity(prediction, target, dim=1).mean()
```

代码/语法	解释
prediction	student 对图片输出的 embedding。
target	teacher 目标 embedding，已投影到 student 维度。
F.normalize(...)	保险起见再次归一化，确保比较的是方向。
F.cosine_similarity(..., dim=1)	逐张图片计算 student 和 teacher 的 cosine。
.mean()	对 batch 里的所有样本求平均。
1.0 - cosine	相似度越高 loss 越低。cosine=1 时 loss=0。

relation_loss：一批图片关系对齐

```
def relation_loss(prediction: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    if prediction.shape[0] < 2:
        return prediction.new_tensor(0.0)
    prediction = F.normalize(prediction, p=2, dim=1)
    target = F.normalize(target, p=2, dim=1)
    return F.mse_loss(prediction @ prediction.T, target @ target.T)
```

代码/语法	解释
prediction.shape[0] < 2	batch 少于 2 张时，没有样本之间的关系可比较。
prediction.new_tensor(0.0)	创建和 prediction 同设备/同 dtype 的 0，避免 CPU/GPU/MPS 设备不一致。
prediction @ prediction.T	@ 是矩阵乘法。这里得到 student batch 内两两相似度矩阵。
target @ target.T	得到 teacher 目标的两两相似度矩阵。
F.mse_loss(...)	让 student 的关系矩阵接近 teacher 的关系矩阵。

teacher_topk_listwise_loss：学习老师的 topK 排序

```
teacher_scores = reference_teacher.index_select(0, sample_indices) @ reference_teacher.T
teacher_scores[row_indices, sample_indices] = torch.finfo(teacher_scores.dtype).min
top_teacher_scores, top_indices = torch.topk(teacher_scores, k=effective_top_k, dim=1)
student_scores = prediction @ reference_targets.T
top_student_scores = student_scores.gather(1, top_indices)
teacher_probs = F.softmax(top_teacher_scores / teacher_temperature, dim=1).detach()
student_log_probs = F.log_softmax(top_student_scores / student_temperature, dim=1)
return F.kl_div(student_log_probs, teacher_probs, reduction="batchmean")
```

代码/语法	解释
reference_teacher	全训练集的 teacher embedding 银行。
index_select(0, sample_indices)	取出当前 batch 样本对应的 teacher embedding。
@ reference_teacher.T	计算当前样本与全库样本的 teacher 相似度。
teacher_scores[row_indices, sample_indices] = min	把自己和自己的相似度排除，否则 top1 永远是自己。
torch.topk(...)	取老师认为最相似的 topK 候选。v9 teacher_top_k=12。
student_scores = prediction @ reference_targets.T	计算 student 对全库目标的相似度。
gather(1, top_indices)	只取老师 topK 那些位置上的 student 分数。
softmax(... / temperature)	把分数变成概率分布。temperature 控制分布尖锐程度。
.detach()	老师概率不参与反向传播，只作为目标。
F.kl_div(...)	让 student 的 topK 排序分布接近 teacher。

hard_negative_margin_loss：处理难负样本

hard negative 是看起来很像但不应该排在正样本前面的照片。例如同是静物，食物和生活物件可能构图很像；同是风景，自然和城市也可能相似。hard negative loss 要求正样本分数至少比难负样本高出 margin。

```
positive_mask = reference_bank.scene_labels == anchor_scene
positive_mask[anchor_index] = False
negative_mask = reference_bank.scene_labels != anchor_scene
if same_photo_type_only:
    negative_mask = negative_mask & (reference_bank.photo_type_labels == anchor_photo_type)
positive_indices = torch.nonzero(positive_mask, as_tuple=False).flatten()
negative_indices = torch.nonzero(negative_mask, as_tuple=False).flatten()
losses.append(F.relu(margin - positive_scores[:, None] + negative_scores[None, :]).mean())
```

代码/语法	解释
positive_mask	同 scene_category 的样本是正样本候选。
positive_mask[anchor_index] = False	把自己排除，不拿自己当正样本。
negative_mask	不同 scene_category 的样本是负样本候选。
same_photo_type_only	v9 使用同 photo_type 内的 hard negative，更聚焦容易混淆的样本。
torch.nonzero(...).flatten()	把 mask 中为 True 的位置转成一维 index 列表。

代码/语法	解释
positive_scores[:, None]	给正样本分数增加一个维度，用于和所有负样本两两比较。
negative_scores[None, :]	给负样本分数增加一个维度。
margin - positive + negative	如果 negative 太高或 positive 不够高，就产生正损失。
F.relu(...)	小于 0 的部分变 0，表示已经满足 margin 不再惩罚。

v9 的损失权重

损失项	权重	作用
embedding	1.0	单张 student embedding 对齐 teacher 目标
relation	0.35	batch 内相似关系对齐
listwise	0.06	学习 teacher topK 排序
hard_negative	0.04	拉开难负样本
photo_type	0.08	辅助学习大类
scene	0.16	辅助学习细场景

本章自测

1. cosine_loss 主要对齐什么?
2. relation_loss 为什么需要 batch 至少 2 张?
3. listwise loss 学的是单个 embedding 还是 topK 排序分布?
4. hard negative 为什么对 v10 继续提升重要?

第 10 章 template index：落盘、校验和检索

模型只负责把图片变成 embedding。真正推荐模板，需要把所有模板 embedding 做成 index。当前 v9 static int8 ONNX 候选 index 目录是 tools/preprocessed/template_index.new_batch.student-tiny-cnn-v9-static-int8-onnx-512/。

index 目录文件

文件	作用
templates.embeddings.f32	728 条 512 维 float32 embedding 二进制矩阵
templates.keys.json	数字 key 与 template_id/photo_type/scene_category 的映射
templates.metadata.jsonl	App 可用模板元数据
templates.usearch.idx	USearch 原型索引
templates.usearch.json	USearch manifest，记录 count、dimension、metric
templates.quality_report.md/json	质量评估报告

构建 index 的核心代码

```
def load_ready_vectors(metadata_path: Path, expected_dimension: int = 256) -> list[TemplateVector]:
    vectors: list[TemplateVector] = []
    seen_template_ids: set[str] = set()
    for line_number, record in enumerate(load_jsonl(metadata_path), 1):
        core_errors = validate_template_metadata.validate_core(record)
        if core_errors:
            raise ValueError(f"line {line_number}: {'; '.join(core_errors)}")
        template_id = record["template_id"]
        if template_id in seen_template_ids:
            raise ValueError(f"line {line_number}: duplicate template_id: {template_id}")
        seen_template_ids.add(template_id)
```

代码/语法	解释
expected_dimension: int = 256	默认维度是旧原型 256。构建 v9 index 时必须传 --dimension 512。
vectors = []	准备收集所有 ready embedding。
seen_template_ids = set()	用集合记录见过的 template_id，防止重复。
enumerate(..., 1)	逐行读取 metadata，并从 1 开始计行号，报错更友好。
validate_core(record)	校验 metadata 核心字段。
if core_errors: raise ValueError	只要核心字段有错就停止构建，避免坏 index。
record["template_id"]	强制取字段，缺字段会 KeyError，说明 metadata 不合格。
template_id in seen_template_ids	检查重复模板 ID。
seen_template_ids.add(template_id)	通过检查后加入集合。

维度和 embedding ready 校验

```
embedding = record["embedding"]
if embedding.get("status") != "ready":
    raise ValueError(f"line {line_number}: embedding for {template_id} is not ready")
if embedding.get("dimension") != expected_dimension:
    raise ValueError(
        f"line {line_number}: expected embedding dimension {expected_dimension}, "
        f"got {embedding.get('dimension')}"
    )
```

代码/语法	解释
embedding.get("status") != "ready"	没有 ready 的 embedding 不能进入 index。
embedding.get("dimension") != expected_dimension	维度必须严格匹配。v9 static int8 ONNX 是 512。
raise ValueError(...)	发现错误立刻失败，不允许自动修。
为什么严格	维度不同的向量无法做有效 dot product，混用会让推荐结果没有意义。

validate index 的底层检查

```
payload = path.read_bytes()
expected_bytes = count * dimension * 4
if len(payload) != expected_bytes:
    raise ValueError(
        f"{path} expected {expected_bytes} bytes for {count}x{dimension} f32, got {len(payload)}"
    )
values = struct.unpack(f"<{count * dimension}f", payload)
```

代码/语法	解释
path.read_bytes()	一次性读取二进制 embeddings 文件。
count * dimension * 4	理论字节数。float32 每个数 4 字节。
len(payload) != expected_bytes	文件大小不对，说明 count、dimension 或文件内容有问题。
struct.unpack(...)	把二进制字节还原成 Python 浮点数。
f"<{count * dimension}f"	小端字节序，读取 count*dimension 个 float32。

USearch 和 exact fallback

USearch/ANN 是为了快速近邻检索。现在 C++ 端已有 exact dot-product fallback，意思是即使 USearch backend 未生产接入，也可以用逐个比较的方式验证功能。728 条模板很小，exact fallback 在原型阶段足够验证正确性；未来模板库变大后才更依赖 ANN 性能。

本章自测

1. 为什么构建 v9 index 必须传 dimension=512?
2. templates.keys.json 和 templates.metadata.jsonl 顺序为什么必须一致?
3. float32 为什么乘 4 字节?

4. ANN 和 exact fallback 的区别是什么？

第 11 章 质量评估：怎么看 top1、top5、同场景、同类型和问题样本

训练指标只是模型学习情况的一部分。这个产品真正关心的是检索质量：当前画面能不能在模板库里找回合适模板。你要会读 quality_report，而不是只看 loss 降没降。

核心指标解释

指标	零基础解释	产品含义
top1 同场景	第一名是否和查询照片 scene_category 一样	直接用 top1 时的可靠度
top5 同场景	前五名里是否至少有一个同场景	top5 + rerank 的召回基础
同类型	photo_type 是否一致	避免人像推荐风景、食物推荐人像
构图匹配	composition 字段是否相近	辅助线是否适合当前画面
光线匹配	lighting 字段是否相近	提示是否符合光照状态
低 margin	top1/top2 分数差很小	推荐不稳定，需要稳定策略

v9 static int8 ONNX 当前质量

topK	同 photo_type	同 scene_category	构图命中	光线命中
top1	95.33%	88.87%	94.64%	89.01%
top3	97.53%	94.78%	98.21%	94.64%
top5	98.21%	96.15%	98.90%	96.57%

怎么解释这些数据

- top1 同场景 88.87% 不够支撑直接取第一名做最终推荐。
- top5 同场景 96.15% 说明正确候选通常在前五里，所以要 top5 + 规则重排。
- top5 同类型 98.21% 较高，但仍有跨 photo_type top1，需要 App 做 photo_type 过滤或加减分。
- 低 margin 数 270 说明很多查询第一第二名很接近，需要连续稳定领先才切模板。

质量报告里的问题样本

问题样本不是失败羞耻表，而是下一轮训练素材和规则重排的方向。比如 plant_still_life 当前 template candidates 只有 5 条，urban_landscape 只有 29 条，这些弱类别需要继续补素材。

上线判断

条件	v9 当前情况	判断
离线 top5 同场景 >= 95%	96.15% static int8 ONNX	满足首发候选门槛
模型大小 <= 5MB	301,677 bytes	满足
Android 真机 p50/p90	仓库无真机数据	必须 App 接入后实测

条件	v9 当前情况	判断
连续预览稳定	仓库无法验证	必须 10-30 分钟真机跑
正式库合并	new_batch 尚未合并 visual-clean	先灰度，达标后再决定

本章自测

1. 为什么 v9 top5 同场景达标，但不能只用 top1？
2. 低 margin 对 App 交互有什么影响？
3. 问题样本应该用于责怪模型，还是用于指导下一轮数据和训练？
4. 上线判断为什么同时需要离线质量和真机性能？

第 12 章 导出和量化：TorchScript、ONNX、dynamic int8、static int8

从 checkpoint 到端侧包

```
checkpoint.best.pt
  -> student_model.fp32.torchscript.pt
  -> student_model.dynamic_int8.torchscript.pt
  -> student_model.fp32.onnx
  -> student_model.static_int8.onnx
  -> android_onnx_runtime_manifest.json
  -> android_runtime_config.json
```

为什么 dynamic int8 TorchScript 不是 Android 商用量化包

export_student_model.py 里的 dynamic int8 只量化 Linear 层，卷积仍是 FP32；而 v9 模型的大量计算在卷积里。它适合作为 PyTorch 原型产物，不等于 Android 商用量化包。Android 首发选择 ONNX Runtime Mobile 的 static int8 ONNX，因为它静态校准并量化 Conv、Gemm、MatMul。

ONNX 导出代码

```
torch.onnx.export(
    model=model,
    args=dummy,
    f=str(onnx_path),
    input_names=["image"],
    output_names=["embedding"],
    dynamic_axes={
        "image": {0: "batch"},
        "embedding": {0: "batch"},
    },
    opset_version=17,
)
```

代码/语法	解释
model=model	要导出的 PyTorch 模型。
args=dummy	示例输入 tensor，用来追踪模型计算图。dummy shape 是 [1,3,224,224]。
f=str(onnx_path)	输出 ONNX 文件路径。
input_names=["image"]	给输入 tensor 命名，Android 侧 session.run 要用这个名字。
output_names=["embedding"]	给输出 tensor 命名。
dynamic_axes	允许 batch 维动态变化，单张或多张都可推理。
opset_version=17	ONNX 算子集版本。Android runtime 版本要能支持。

static int8 量化代码

```
quantize_static(
    model_input=str(onnx_model),
    model_output=str(output_model),
    calibration_data_reader=calibration_reader,
    quant_format=QuantFormat.QDQ,
    activation_type=QuantType.QUInt8,
    weight_type=QuantType.QInt8,
    per_channel=True,
    calibrate_method=CalibrationMethod.MinMax,
    op_types_to_quantize=["Conv", "Gemm", "MatMul"],
)
```

代码/语法	解释
quantize_static	ONNX Runtime 的静态量化函数。需要真实校准数据。
model_input	FP32 ONNX 输入模型。
model_output	量化后 int8 ONNX 输出模型。
calibration_data_reader	提供真实图片 batch，用来统计激活范围。
QuantFormat.QDQ	量化格式，用 Quantize/DeQuantize 节点表达 int8 计算。
activation_type=QUInt8	激活值用无符号 int8。
weight_type=QInt8	权重用有符号 int8。
per_channel=True	按通道量化权重，通常精度更好。
CalibrationMethod.MinMax	用校准数据的最小最大值估计量化范围。
["Conv","Gemm","MatMul"]	量化卷积、全连接/矩阵乘等主要计算算子。

量化后必须重新评估

量化会改变模型数值。不能只看文件变小。必须用量化模型重新生成 template embedding、重建 index、跑 quality_report。本项目 v9 FP32 top5 同场景 96.57%，static int8 ONNX top5 同场景 96.15%，下降可接受，所以可作为 Android 首发候选。

本章自测

1. TorchScript 和 ONNX 都是模型格式，它们的 Android 首发地位一样吗?
2. dynamic int8 为什么不等于完整商用量化包?
3. static int8 为什么需要 calibration data?
4. 量化后为什么要重新生成 embedding 和 index?

第 13 章 Android 接入：你不写 App，也要会验收

交付文件

文件/目录	必须交付原因
student_model.static_int8.onnx	Android 首发端侧 embedding 模型
android_onnx_runtime_manifest.json	输入、输出、量化、sha256、验证信息
android_runtime_config.json	抽帧、topK、rerank、预算等接入配置
student_model.fp32.onnx	排查 int8 精度问题的对照模型
template_index/	v9 static int8 ONNX 生成的 728 条 512 维模板 index

ONNX Runtime 推理验收

- ONNX Runtime 能加载 student_model.static_int8.onnx。
- 输入 tensor 名字是 image，shape 是 [1,3,224,224]，dtype 是 float32。
- 输出 tensor 名字是 embedding，shape 是 [1,512]，dtype 是 float32。
- 输出 embedding L2 norm 约等于 1.0。
- 同一张测试图多次推理结果稳定。
- Python ONNX 与 Android ONNX 对同一张图输出 cosine 足够接近。

为什么不能每帧跑 embedding

相机预览通常 30 FPS。若每帧跑一次 embedding，就等于每秒推理 30 次。即使单次推理 80ms，也已经超过一帧 33ms 的预算，会堆积、卡顿、发热、耗电。首版应按 500ms-1000ms 抽帧，或画面明显变化时触发，并且 single-flight：上一次 embedding 没跑完时丢弃新帧。

top5 rerank 规则

规则	分值	说明
photo_type 匹配	+0.08	优先保证人像/风景/静物大类不串
高置信 scene_category 匹配	+0.06	场景明确时提升同场景候选
构图类型匹配	+0.03	让辅助线更贴合画面
光线类型匹配	+0.02	让光线提示更合理
photo_type 明显不匹配	-0.08	惩罚跨大类错误
高置信 scene 不匹配	-0.04	降低错场景候选

问 Android 队友的问题

1. 目标设备型号和芯片是什么?
2. ONNX Runtime Android 版本锁定到多少? 是否 1.19.x?
3. 模型和 index 是否从 assets 复制到 app-private files 目录?

4. 实际 embedding 触发间隔是多少？有没有 single-flight？
5. 目标设备 p50/p90 是多少？是否满足 p50 <= 80ms、p90 <= 150ms？
6. 连续预览 10-30 分钟内存、温度、FPS 是否稳定？
7. top5 是否真的进入规则重排？低 margin 时是否保持稳定模板？

本章自测

1. Android assets 为什么通常要复制到 app-private files 目录？
2. single-flight 是什么意思？
3. 如果 App 只取 top1，会违背哪个产品策略？
4. 真机 p50/p90 为什么不能用电脑离线测试替代？

第 14 章 v9 首发条件和 v10 迭代方向

v9 可以首发候选的条件

- 离线质量达标：static int8 ONNX top5 同场景 96.15%，超过 95% 门槛。
- 模型大小达标：301,677 bytes，小于 5MB。
- 交付格式明确：ONNX Runtime Mobile static int8 ONNX。
- index 配套：728 条、512 维、由同一 static int8 ONNX 生成。
- App 使用 top5 + 规则重排，且 500ms-1000ms 抽帧。

v9 还不能宣称完整商用默认的原因

- 缺 Android App 工程接入验证。
- 缺目标设备 p50/p90、内存、发热、连续预览稳定性数据。
- 训练集只有 579 条，类别不均衡。
- plant_still_life 和 urban_landscape 覆盖偏少。
- new_batch 尚未合并为正式 visual-clean。

什么时候必须继续 v10

触发条件	v10 方向
真机 p90 > 150ms 或发热严重	进一步减小模型、降低输入、优化 runtime 或抽帧策略
top1 跨类型错误影响体验	加强 photo_type 辅助头、rerank、hard negative
弱类别召回差	补 plant、urban、portrait outdoor/couple/group 数据
低 margin 导致 UI 抖动	训练上继续 listwise/hard negative，App 上稳定策略
量化后质量下降过大	改 calibration 数据、量化策略或模型结构
模板库扩到 3000+	重建 teacher/student/index，重新评估 ANN 和 exact 差异

数据补充优先级

1. portrait：outdoor、urban、couple、group。
2. plant_still_life：当前候选极少，是明显弱类。
3. urban_landscape：城市风景需要增加多样性。
4. hard negative：同场景但构图/光线不同，或同类型但场景不同的难样本。
5. 目标：先把可训练 template candidates 提升到 3000+，再冲 5200 完整目标。

本章自测

1. v9 能进入 App 接入和灰度，为什么还不能直接说正式商用默认？
2. v10 的第一优先数据补充是什么？
3. 如果 App 反馈推荐频繁跳变，你会同时从模型和 App 哪两侧处理？
4. 为什么 new_batch 达标后也不能无条件污染 visual-clean？

附录 A 固定命令和验收清单

全量验证命令

```
cd /Volumes/桌面/AI-camera/tools/build_template_db
python3 -m unittest discover -s tests

cd /Volumes/桌面/AI-camera
python3 tools/build_template_db/check_pipeline.py --root . --require-embeddings

cd /Volumes/桌面/AI-camera/engine
cmake --build build
./build/engine_tests
```

模型包验收

- student_model.static_int8.onnx 存在，大小约 301,677 bytes。
- sha256 为 752bfef24b553a27fb0d419e939a6fc2583a22efda1287b01ce1184409f3e5d9。
- android_onnx_runtime_manifest.json verification.status 为 ready。
- 输出 shape 为 [batch,512]，int8_output_l2_norm 约 1.0。
- static int8 ONNX quality_report top5 同场景 96.15%。

index 验收

- templates.keys.json count=728、dimension=512。
- templates.embeddings.f32 大小为 1,490,944 bytes。
- templates.metadata.jsonl 行数为 728。
- validate_template_index.py 返回 0。
- USearch manifest metric 为 cos，self_check 为 top1。

Android 验收

- 模型和 index 从 assets 复制到 app-private files 目录。
- ONNX Runtime 能加载 static int8 ONNX。
- 预处理严格一致：rotation、RGB、224、NCHW、归一化。
- embedding 实际触发间隔 500ms-1000ms 或画面变化触发。
- p50 <= 80ms，p90 <= 150ms。
- 连续 10-30 分钟无明显内存增长、过热、卡死。
- top5 进入规则重排，低 margin 稳定策略生效。

附录 B 术语表

术语	解释
embedding	图片在语义空间里的数字坐标，用来算相似度
teacher	大模型老师，本项目是 SigLIP2
student	端侧小模型，本项目首发候选是 v9
distillation	让 student 学 teacher 的 embedding 和排序关系
L2 normalize	把向量长度变成 1，方便用 dot product 算 cosine
cosine similarity	衡量两个向量方向是否接近
topK	取相似度最高的 K 个候选
margin	top1 和 top2 分数差，差小表示不确定
hard negative	很像但不该排在前面的负样本
listwise	学习一组候选的排序分布，而不是只学单点
ONNX	跨框架模型交换/部署格式，Android 可用 ONNX Runtime 跑
static int8	用校准数据提前确定量化范围的 int8 量化
dynamic int8	运行时动态量化，项目中仅 Linear 层，不是 Android 首发商用包
USearch/ANN	近邻检索索引，用来快速找相似 embedding
metadata	模板说明书，App 用它生成辅助线和提示

附录 C 10 天自学安排

天数	主题	读文件	练习
Day 1	项目全景	AI引擎设计/工作进度.md；技术设计与工作流程.md	画出素材到 Android runtime 的链路图
Day 2	Python/JSONL/路径	build_template_index.py；validate_template_index.py	解释一条 metadata 和一个 f32 文件大小
Day 3	图片预处理	generate_onnx_embeddings.py；09-AI接入包说明.md	写出 RGB/NCHW/normalize 的含义
Day 4	embedding/cosine/topK	evaluate_template_index_quality.py	手算二维向量 normalize 和 dot product
Day 5	student 模型结构	student_embedding_model.py	逐句解释 forward 和 DepthwiseSeparableBlock
Day 6	蒸馏训练	train_student_embedding.py	解释 cosine/relation/listwise/hard negative
Day 7	teacher 和数据集	generate_hf_image_embeddings.py；build_distillation_dataset.py	说明 teacher 为什么不能直接端侧商用
Day 8	导出和量化	export_student_model.py；quantize_onnx_static.py	区分 TorchScript dynamic int8 和 ONNX static int8
Day 9	Android 接入验收	09-AI接入包说明.md；android_runtime_config.json	写一份问 Android 队友的问题清单
Day 10	上线和 v10 决策	quality_report；需要用户完成的任务.md	根据指标判断 v9 go/no-go 和 v10 数据方向